# Bash Tape Deck

## Shell tutorials for people who hate slide decks

### Why I (**Paulo Jerônimo**) Built This Open Source Project

**I wanted a way to demonstrate, step by step, the execution of a set of scripts I had written to explain the concept of Agent Skills**. Those scripts were already in good shape and did exactly what I wanted. The problem was presentation.

Shell scripts are tedious to explain to an audience that does not know much Bash. I also did not want to use slides, and I definitely did not want to write yet another tutorial document like the many I had already produced before.

What I wanted was something more elegant: an environment where I could put the scripts in motion, let the commands do real work, and still move backward during the presentation to show what had just happened.

That led me to the idea of building an execution environment, using Docker or LXD, where I could replay the same scripted demonstration as many times as necessary and always return to a known state.

I had already used similar ideas before, but in cloud-hosted environments. One example was Katacoda, which was later discontinued by O'Reilly.

**Bash Tape Deck** is the local, script-driven version of that idea. But it can also be executed easily in the cloud through Development Containers. That means you can run it on practically any cloud environment and, even more easily, on GitHub Codespaces. See the video reference for that setup.

# 1. What This Project Does

- Runs steps defined in `project/steps.N.sh` files.
- Shows each step title centered and numbered.
- Prints each command before execution.
- Supports heredoc commands inside step files.
- Uses interactive navigation (`Enter`, `b`, `q`) between rendered slides.
- Shows a final centered `Done!` summary with total executed steps.

## 1.1. Tape Deck Analogy

Bash Tape models a shell tutorial as if it were a tape deck:

- `play`: start the tape
- `pause`: wait on each slide for the next action
- `back`: move to the previous rendered slide
- `forward`: move to the next slide
- `rewind`: undo a given `steps.N.sh` through its matching `steps.N.rewind.sh`
- `stop`: leave playback without ejecting the tape
- `eject`: finalize the tape and run `steps.end.sh`, if present

This distinction matters when a tape changes local state. Stopping preserves the current environment so the tape can be resumed later. Ejecting is the explicit cleanup action.

# 2. Current Capabilities

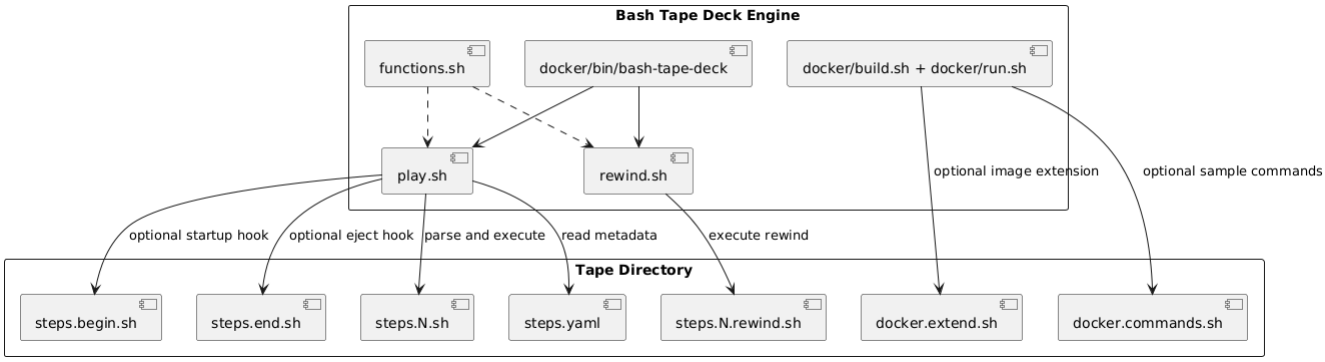
1. Interactive playback
   - Bash Tape Deck plays shell tutorials from `steps.N.sh` files.
   - Each slide can show a title, explanatory metadata, commands, and command output.
   - Commands are executed for real while being presented.
2. Tape-style navigation
   - You can move forward and back through rendered slides.
   - Previously visited slides are shown from cache and are not re-executed.
   - The player supports `stop` and `eject` as distinct operations.
3. Script-friendly authoring
   - The engine understands common tutorial patterns such as heredoc-based file creation.
   - This makes it practical to present scripts that generate other scripts or configuration files.
4. Lifecycle hooks
   - A tape can define `steps.begin.sh` for setup.
   - A tape can define `steps.end.sh` for explicit cleanup.
   - This is useful when a tutorial temporarily changes local state such as Git configuration.
5. Event logging and state tracking
   - The runtime records tape events in a JSONL log under `.bash-tape-deck/`.
   - This makes playback, rewind, stop, and eject actions inspectable after the run.
6. Explicit rewind support
   - Each `steps.N.sh` can have a matching `steps.N.rewind.sh`.
   - That allows each part of a tape to define its own reset strategy.
7. Local and container execution
   - Bash Tape Deck runs directly on the host.
   - It also runs inside Docker.
   - Its container model is modular, so each tape image can add only the tools it needs.
8. Automated regression coverage
   - The project already includes integration testing for the Docker wrapper and tape logging behavior.
   - This provides a baseline against regressions in core user-facing workflows.
9. Planned evolution
   - The current roadmap for session-aware playback is documented in Next Steps.

# 3. Architecture

Bash Tape Deck is split into a small execution engine and tape-specific content. This separation is what makes the project modular both on the host and inside Docker images.

## 3.1. Component Model

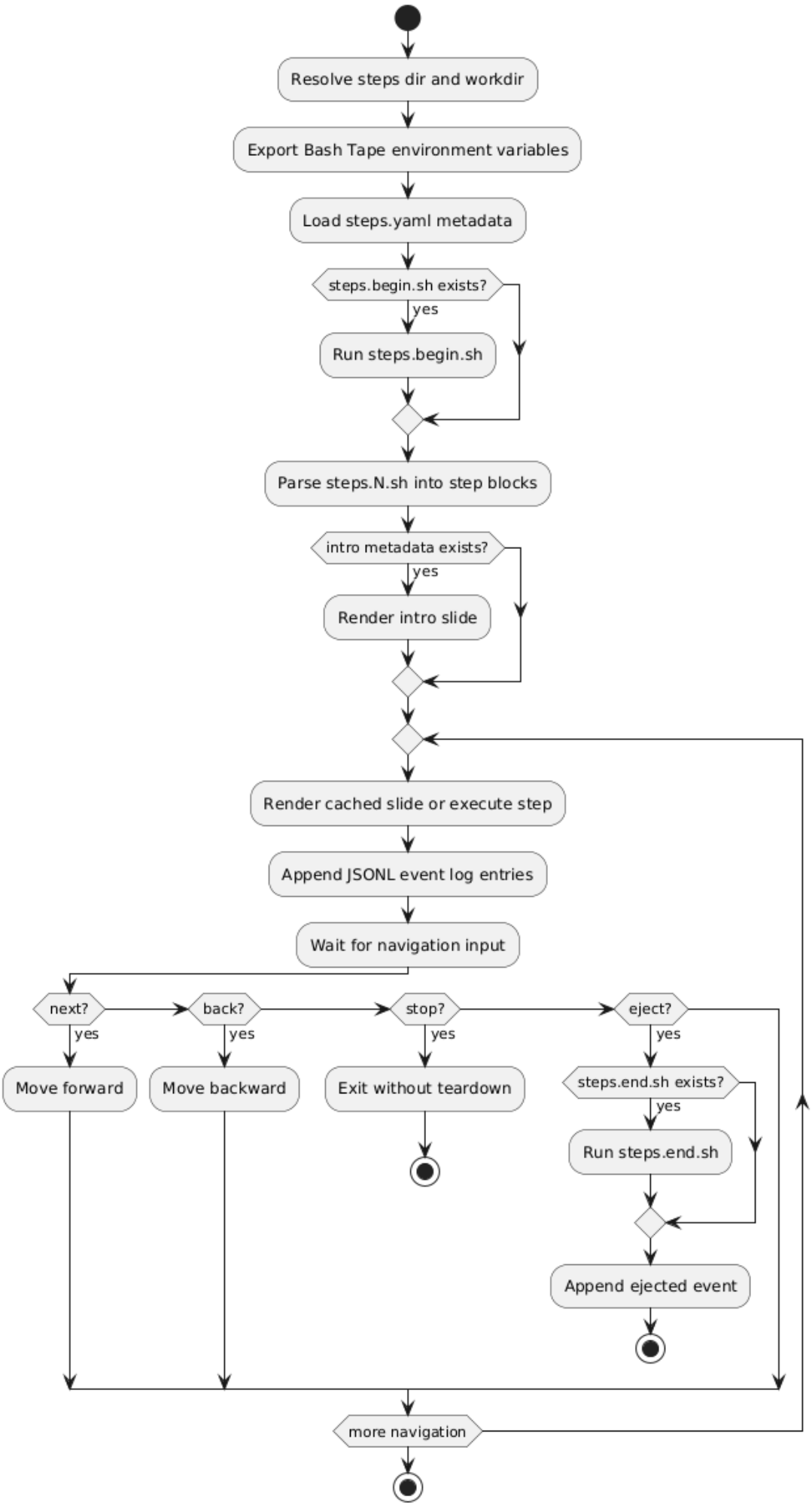
At the highest level, the engine is responsible for parsing steps, rendering slides, handling navigation, and exposing a stable execution environment. Each tape provides only its own content and optional extension points.




## 3.2. Playback Architecture

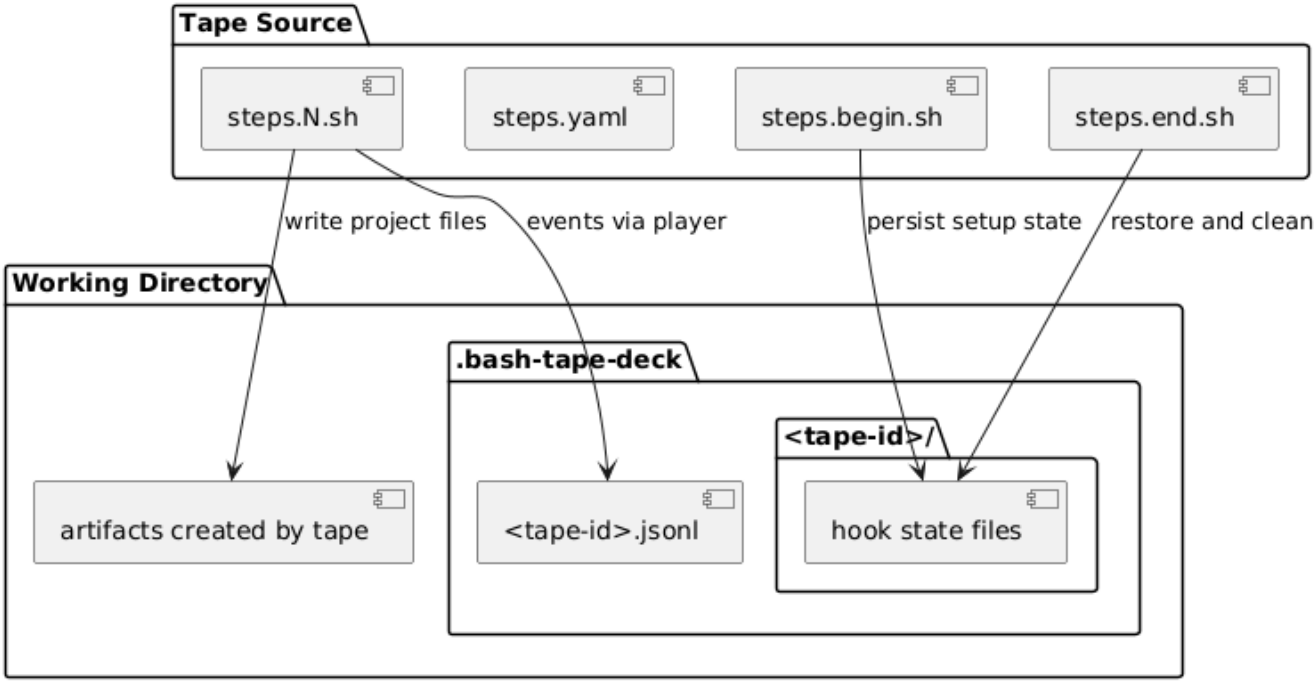
The `play.sh` flow has two independent responsibilities:

- render and navigate slides without re-executing cached steps
- execute tape lifecycle hooks and keep an event log for the tape state


Resolve steps dir and workdir
Export Bash Tape environment variables
Load steps.yaml metadata
steps.begin.sh exists?
yes
Run steps.begin.sh
Parse steps.N.sh into step blocks
intro metadata exists?
yes
Render intro slide
Render cached slide or execute step
Append JSONL event log entries
Wait for navigation input
next?
yes
Move forward
back?
yes
Move backward
stop?
yes
Exit without teardown
eject?
yes
steps.end.sh exists?
yes
Run steps.end.sh
Append ejected event
more navigation

## 3.3. Execution Context

Every tape runs against a working directory that is intentionally separate from the tape source directory. That is what lets a single tape be replayed in a clean workspace, in `build/`, or in a Docker-mounted directory.




The engine exports a standard contract to tape scripts:

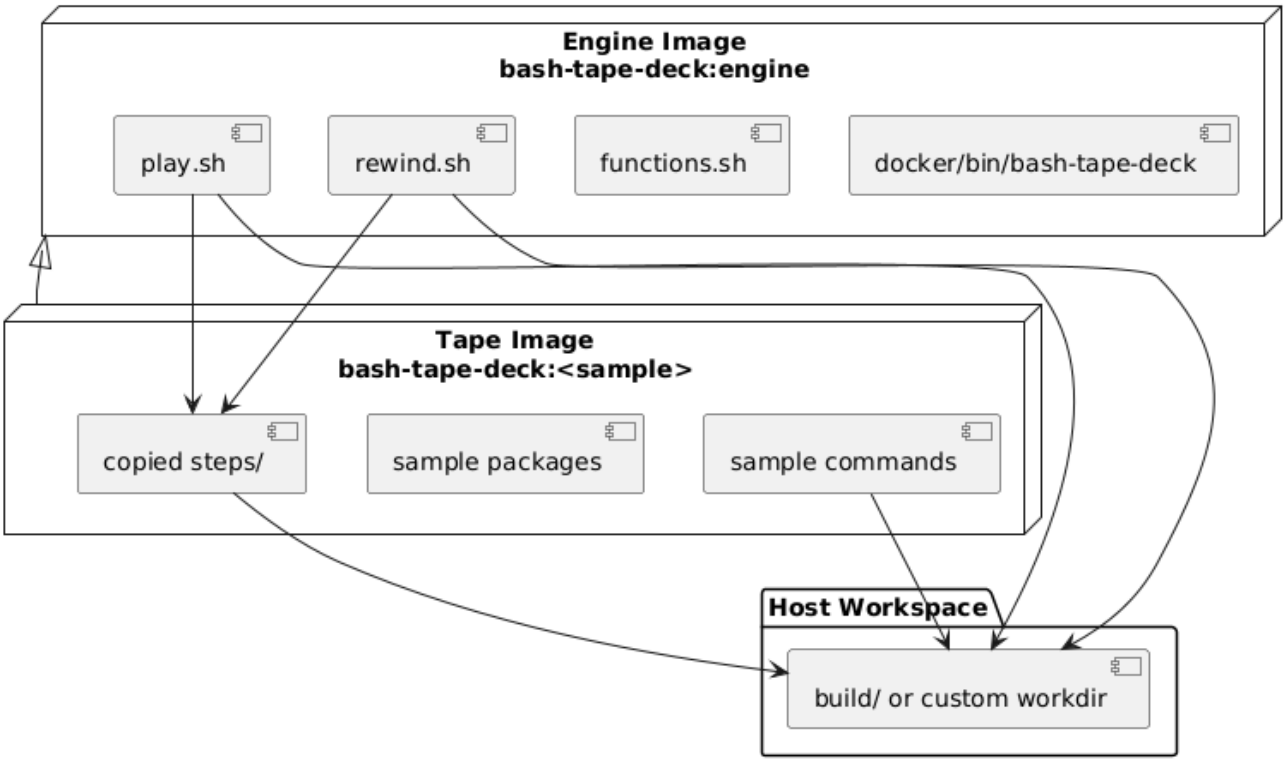
- `BASH_TAPE_ID`
- `BASH_TAPE_DIR`
- `BASH_TAPE_STATE_DIR`
- `BASH_TAPE_LOG_FILE`
- `BASH_TAPE_STEPS_DIR`
- `BASH_TAPE_STEPS_NUMBER`
- `BASH_TAPE_RUN_WORKDIR`

This avoids hardcoded paths in hooks such as `steps.begin.sh` and `steps.end.sh`.

## 3.4. Modular Container Architecture

The Docker model is intentionally layered:

- the engine image contains only the generic Bash Tape Deck runtime
- each tape image copies its own steps and can add only the packages it needs
- optional runtime commands live with the tape, not in the engine




The Docker scripts build this structure in two phases:

1. build `bash-tape-deck:engine`
2. build `bash-tape-deck:<sample>` from the engine
3. apply `docker.extend.sh` from the tape, if present
4. copy the tape steps into the sample image
5. execute `play.sh`, `rewind.sh`, or tape-specific commands inside the resulting container

This keeps the engine small while still allowing a tape such as a private tutorial to install extra tools like Git, Node.js, or Codex without forcing those dependencies onto unrelated tapes.

# 4. Files

This section is generated by `docs/files.adoc.sh`.

```
Tree                                Description
-----------------------------------  -----------
.devcontainer                       Development Container assets for local and cloud
workspaces.
+-- Dockerfile                      Dev Container image definition for local and
Codespaces development.
+-- devcontainer.json               Dev Container configuration consumed by supported
editors and the devcontainer CLI.
\-- post-start.sh                   Post-start hook that prepares the workspace
defaults inside the Dev Container.
.gitignore                          Root ignore rules for generated and local-only
files.
AGENTS.md                           Repository-specific agent rules for code
generation, testing, and commits.
LICENSE                             MIT license for the project.
README.adoc                         Main project document assembled from the section
files in docs/.
cassette-art.sh                     Public cassette artwork wrapper around
cassette_art_main.
diagrams                            PlantUML source diagrams used by the
documentation.
+-- architecture-components.puml    PlantUML diagram describing the main Bash Tape
components.
+-- architecture-docker.puml        PlantUML diagram describing the Docker-oriented
runtime flow.
+-- architecture-playback-flow.puml PlantUML diagram describing playback behavior and
navigation.
\-- architecture-workdir.puml       PlantUML diagram describing workspace and state
directories.
docker                              Docker runtime assets and wrapper scripts.
+-- Dockerfile                      Runtime Docker image definition used by the Bash
Tape engine.
+-- bin                             Container entrypoint helpers.
|   \-- bash-tape                   Container entrypoint that dispatches to play and
rewind.
+-- build.sh                        Wrapper that builds the engine image and sample-
specific images.
+-- common.sh                       Shared functions and defaults used by the Docker
wrapper scripts.
\-- run.sh                          Wrapper that runs tapes and sample-specific
container commands.
docs                                AsciiDoc source files that compose the main
README.
+-- _                               Shared documentation fragments.
|   \-- attributes.adoc             Shared AsciiDoc attributes reused across the
documentation set.
```

```
+-- architecture.adoc                Architecture section of the README.
+-- author.adoc                      Author section of the README.
+-- current-capabilities.adoc        Current capabilities section of the README.
+-- current-parser-limitations.adoc  Documentation of the current parser constraints
and tradeoffs.
+-- development-model.adoc           Development model section of the README.
+-- files.adoc                       Literal wrapper that includes docs/files.txt in
the README.
+-- files.adoc.sh                    Generator for docs/files.txt and docs/files.adoc.
+-- files.txt                        Generated two-column file inventory rendered
inside docs/files.adoc.
+-- license.adoc                     License section of the README.
+-- navigation.adoc                  Navigation section of the README.
+-- next-steps.adoc                  Planned work and roadmap section of the README.
+-- references.adoc                  Reference links section of the README.
+-- requirements.adoc                Requirements section of the README.
+-- step-file-format.adoc            Documentation for the tape step file format.
+-- tests.adoc                       Testing section of the README.
+-- usage.adoc                       Usage section of the README.
\-- what-this-project-does.adoc      Project overview section of the README.
functions.sh                         Shared shell functions for playback, rendering,
logging, and cassette art.
images                               Image assets referenced by the documentation.
\-- art.gif                          Animated artwork displayed at the top of the
GitHub README.
\-- art.png                          Rendered artwork displayed at the top of the
README.
play.sh                              Interactive tape player for local execution.
rewind.sh                            Tape rewind dispatcher for local execution.
samples                              Public sample tapes shipped with the project.
+-- .gitignore                       Ignore rules for generated content under public
samples.
\-- fibonacci                        Sample tutorial that teaches Bash through
Fibonacci implementations.
|   +-- .gitignore                   Ignore rules for generated files in the Fibonacci
sample.
|   +-- docker.extend.sh             Sample-specific Docker extensions for the
Fibonacci tape.
|   +-- steps.1.rewind.sh            Rewind logic for the first Fibonacci tape.
|   +-- steps.1.sh                   First Fibonacci tape.
|   +-- steps.2.rewind.sh            Rewind logic for the second Fibonacci tape.
|   +-- steps.2.sh                   Second Fibonacci tape.
|   +-- steps.3.rewind.sh            Rewind logic for the third Fibonacci tape.
|   +-- steps.3.sh                   Third Fibonacci tape.
|   \-- steps.yaml                   Metadata and narration content for the Fibonacci
tapes.
tests                                Automated test suite and fixtures.
+-- fixtures                         Static test fixtures used by integration tests.
    \-- docker-sample                Fixture tape used to exercise the Docker wrapper
end to end.
    |   +-- steps.1.sh               Fixture tape used by Docker integration tests.
```

```
    |   +-- steps.begin.sh           Fixture initialization hook used by Docker
integration tests.
    |   +-- steps.end.sh             Fixture cleanup hook used by Docker integration
tests.
    |   \-- steps.yaml               Fixture metadata used by Docker integration
tests.
+-- integration                      Integration tests that validate the main user
flows.
    +-- test-devcontainer-config.sh  Integration test that validates the Dev Container
configuration files.
    \-- test-docker-wrapper.sh       Integration test that exercises the Docker
wrappers end to end.
\-- run.sh                           Test runner that executes the current automated
test suite.
```

# 5. Requirements

## 5.1. For usage via scripts

- A Bash terminal
- `tree`
- `git`
- `yq`
- `batcat`
- `figlet`
- `toilet`

> To check the complete list of requirements, read the docker/Dockerfile

## 5.2. For usage via docker

- A Bash terminal
- `docker`

# 6. Usage

## 6.1. Via scripts

### 6.1.1. Play

Run steps 1 (default):

```
./play.sh samples/fibonacci
```

Run a specific steps file:

```
./play.sh samples/fibonacci 2
```

This expects `samples/fibonacci/steps.2.sh` to exist.

Metadata for each steps file is loaded from `samples/fibonacci/steps.yaml` (multi-document YAML) using `yq`.

Optional lifecycle hooks can be placed next to the steps files:

- `steps.begin.sh`: runs once before the tape starts
- `steps.end.sh`: runs only when the tape is explicitly ejected

These hooks run in the same working directory used by the tape itself. They are useful for setup and cleanup tasks such as temporarily changing local configuration and restoring it afterward.

The player also exports a small set of environment variables so tape scripts can discover their execution context without hardcoded paths:

- `BASH_TAPE_ID`: stable tape identifier derived from the steps directory
- `BASH_TAPE_DIR`: base directory for Bash Tape Deck internal data in the current working directory
- `BASH_TAPE_STATE_DIR`: tape-specific directory for hook/script state files
- `BASH_TAPE_LOG_FILE`: JSONL event log for the tape
- `BASH_TAPE_STEPS_DIR`: absolute path to the tape directory
- `BASH_TAPE_STEPS_NUMBER`: current `steps.N.sh` number being played or rewound
- `BASH_TAPE_RUN_WORKDIR`: effective working directory used by the tape

When a tape needs to persist auxiliary files, `BASH_TAPE_STATE_DIR` is the intended location.

Lifecycle flow:

- `steps.begin.sh` runs automatically when the tape starts.
- `s` stops the player without running `steps.end.sh`.

- `e` ejects the tape and runs `steps.end.sh`.
- If `steps.end.sh` exists and you stop without ejecting, Bash Tape Deck warns that cleanup was skipped and the environment may remain inconsistent.

By default, step commands run in `build/` at the project root. You can override this by setting `BASH_TAPE_RUN_WORKDIR`:

```
BASH_TAPE_RUN_WORKDIR=/tmp/demo ./play.sh samples/fibonacci 2
BASH_TAPE_RUN_WORKDIR=/tmp/demo ./rewind.sh samples/fibonacci 2
```

You can also export the variable once before running multiple commands:

```
export BASH_TAPE_RUN_WORKDIR=/tmp/demo
./play.sh samples/fibonacci 2
./rewind.sh samples/fibonacci 2
unset BASH_TAPE_RUN_WORKDIR
```

If you export the variable for temporary testing, it is a good idea to remove it afterward with `unset BASH_TAPE_RUN_WORKDIR`.

> `BASH_TAPE_RUN_WORKDIR` is also honored by `./docker/run.sh play.sh …` and `./docker/run.sh rewind.sh …`. See Via Docker for the wrapper behavior and default working directory.

### 6.1.2. Rewind

Run default reset logic (step 1):

```
./rewind.sh samples/fibonacci
```

Run step-specific reset logic:

```
./rewind.sh samples/fibonacci 2
```

This executes `samples/fibonacci/steps.2.rewind.sh`.

Each `steps.N.rewind.sh` owns the reset strategy required by `steps.N.sh`.

## 6.2. Via Docker

Build the engine image:

```
./docker/build.sh
```

Build a modular image for a specific tape:

```
./docker/build.sh samples/fibonacci
```

Rebuild the engine first, then build the tape image:

```
./docker/build.sh --all samples/fibonacci
```

The `--all` option is only meaningful when a tape directory is provided. It forces a rebuild of the engine image before building the tape image.

This creates `bash-tape-deck:fibonacci` with:

- engine scripts (`play`, `rewind`, helpers)
- steps from `samples/fibonacci`
- only sample-specific packages declared in `samples/fibonacci/docker.extend.sh`

Run the modular image directly:

```
docker run --rm -it bash-tape-deck:fibonacci play
docker run --rm -it bash-tape-deck:fibonacci play 2
docker run --rm -it bash-tape-deck:fibonacci rewind
docker run --rm -it bash-tape-deck:fibonacci rewind 2
```

Without a bind mount, generated files are created under `/workspace` inside the container and are lost when the container exits. This remains the default internal working directory when running the sample image directly with `docker run`.

To control where the working files are created on the host, mount a directory to `/workspace`:

```
docker run --rm -it -u "$(id -u):$(id -g)" -v "$PWD:/workspace" \
  bash-tape-deck:fibonacci play
```

In this example, files such as `fibonacci-lab` are created in the current host directory.

When using `./docker/run.sh play.sh ⋯` or `./docker/run.sh rewind.sh ⋯`, the wrapper mounts a host directory as `/workspace` automatically. By default, that host directory is `build/` in the project root.

This means that, unless you override it, files generated by the wrapper are created under `build/` at the project root.

With the wrapper, you can use the same variable name to override that host-side working directory:

```
BASH_TAPE_RUN_WORKDIR=/tmp/demo ./docker/run.sh play.sh samples/fibonacci 2
```

In that case, the wrapper mounts `/tmp/demo` as `/workspace`, and generated files are created there.

You can also run through the wrapper:

```
./docker/run.sh play.sh samples/fibonacci
./docker/run.sh play.sh samples/fibonacci 2
./docker/run.sh rewind.sh samples/fibonacci
./docker/run.sh rewind.sh samples/fibonacci 2
```

## 6.3. Via Development Containers

The repository includes a standard Dev Container definition based on an official Dev Containers base image plus the official Docker-in-Docker feature. Its purpose is operational, not architectural: it lets you use the existing project commands inside a Development Container or a GitHub Codespace without changing how Bash Tape Deck works.

The configured environment keeps the current conventions:

- the repository is mounted at `/workspaces/<repository-name>`
- the default tape workdir is `/workspaces/<repository-name>/build`
- `BASH_TAPE_RUN_WORKDIR` still overrides the default workdir
- Docker access is provided by the Dev Container Docker feature

To use it locally with Development Containers through an editor integration:

1. Install a compatible Development Containers environment. For example, Visual Studio Code with the Dev Containers extension.
2. Open this repository in that environment.
3. Choose the command that reopens the project inside the Dev Container.
4. Wait for the container to build and for the terminal to become available.
5. Run the same project commands you would run on the host.

To use it locally through the `devcontainer` CLI:

1. Install the `devcontainer` CLI.
2. From the repository root, start the environment:

   ```
   devcontainer up --workspace-folder .
   ```

3. Open a shell inside the running Dev Container:

```
devcontainer exec --workspace-folder . bash
```

4. Run the same project commands you would run on the host.

To use it remotely with GitHub Codespaces:

1. Create a Codespace for this repository.
2. Wait for the configured Dev Container to finish starting.
3. Open the integrated terminal.
4. Run the same project commands you would run locally.

In both cases, you should be able to use commands such as:

```
./play.sh samples/fibonacci
./rewind.sh samples/fibonacci 2
./docker/build.sh samples/fibonacci
./docker/run.sh play.sh samples/fibonacci
```

The intent is that Development Containers and Codespaces change only the execution environment, not the Bash Tape Deck command line or its workdir conventions.

# 7. Navigation

During playback:

- `Enter`: next step
- `b`: go back to previous step
- `e`: eject the tape and run `steps.end.sh`, if present
- `s`: stop playback without ejecting the tape

When navigating back/forward, previously visited steps are rendered from cache and are not re-executed.

## 7.1. Step File Format

File: `samples/fibonacci/steps.N.sh`

- Shebang is allowed and ignored by `play.sh`.
- A step title is a comment line: `# Step title…`
- One or more commands follow the comment.
- A new slide is created only when a step block is terminated by a blank line.
- If multiple commands appear without a blank line between step blocks, they remain in the same slide even when they are visually short.
- Heredoc commands are supported.

To split a large file construction across multiple slides, you must create multiple titled step blocks separated by blank lines.

Example:

```
#!/usr/bin/env bash

# Create folder
mkdir -p demo

# Write file with heredoc
cat > demo/file.txt <<'EOF'
hello
EOF
```

# 8. Current Parser Limitations

The current `play.sh` parser is intentionally pragmatic, but it is not a full shell parser. It works well for the Bash Tape authoring style used by this project, yet it still has important limitations that tape authors should understand.

Known limitations include:

- Slide separation is structural, not semantic. A new slide is created only when a titled step block ends with a blank line.
- Splitting a logical operation across multiple shell commands does not create multiple slides unless each block has its own title and blank-line separator.
- Some shell constructs are more fragile than plain sequential commands. In particular, grouped redirections such as `{ ⋯ } > file` are currently more error-prone for the parser than plain `cat > ⋯` and `cat >> ⋯` heredoc commands.
- The parser is designed for readable tutorial scripts, not for arbitrary Bash. Very dense command compositions, unusual multiline constructs, and highly nested shell syntax may still produce poor presentation results even when the shell syntax itself is valid.

In practice, the safest authoring style today is:

- one titled step block per slide
- plain sequential commands
- heredoc-based file generation with moderate block size
- explicit blank lines between slides

This is an area intentionally left open for future improvement.

# 9. Tests

Run the automated test suite with:

```
tests/run.sh
```

At the moment, the suite contains an integration test for the Docker wrapper. It validates the end-to-end flow of:

- building a tape image through `./docker/build.sh --all <steps-dir>`
- running `./docker/run.sh play.sh <steps-dir>`
- recording tape events in the JSONL log
- preserving a tape-specific state directory under `.bash-tape-deck/`

The suite also includes a configuration-level test for the Dev Container and Docker wrapper files. It verifies that the repository keeps the expected Dev Container settings and the expected `docker/` script layout.

The current integration fixture lives under `tests/fixtures/docker-sample`.

## 9.1. Manual Checks

If you want to inspect the tape events manually while using the Docker wrapper, you can point execution to a temporary working directory and then inspect the generated JSONL log.

Example:

```
tmp_dir="$(mktemp -d)"
BASH_TAPE_RUN_WORKDIR="$tmp_dir" ./docker/run.sh play.sh samples/fibonacci
```

After the tape starts or finishes, inspect the generated files:

```
find "$tmp_dir/.bash-tape-deck" -maxdepth 2 -type f | sort
```

Then open the JSONL event log:

```
batcat "$tmp_dir/.bash-tape-deck/"*.jsonl
```

You should see events such as:

- `play_started`
- `step_executed`
- `back`

- `forward`
- `stopped`
- `eject_requested`
- `ejected`
- `rewind_started`
- `rewound`

This is the simplest manual way to confirm what the player and wrapper recorded for a tape run.

# 10. Next Steps

## 10.1. Introduce a Session Model

One of the next architectural improvements for Bash Tape Deck is a first-class session model. Today, a tape run is mostly understood as "play a given `steps.N.sh`". The next stage is to treat a tape as a resumable session that can span multiple `steps.N.sh` files in sequence.

The current direction being considered is:

- `play` should no longer require the user to pass a step number in normal usage. Instead, the player should discover the last `steps.N.sh` that finished successfully and resume from the next logical point.
- If playback stopped in the middle of a `steps.N.sh`, that step should not be considered completed. A later `play` should return to that same `steps.N.sh`.
- Resuming an incomplete `steps.N.sh` should trigger an automatic rewind before replaying it, so the step is restarted from a clean state rather than from a partially modified environment.
- When the player reaches the end of a `steps.N.sh`, it should allow either:
  - `stop`, preserving the session for later continuation
  - continue into the next `steps.N.sh` automatically or through an explicit confirmation
- If the current file is already the last `steps.N.sh` in the tape, the player should make that clear and report that there are no additional steps to play.

This means a future session-aware tape run will need to distinguish at least these conditions:

- no session started yet
- session in progress
- current `steps.N.sh` completed successfully
- current `steps.N.sh` interrupted before completion
- session stopped between completed step files
- session fully ejected

That model will likely build on top of the existing JSONL event log already stored under `.bash-tape-deck/`, but it will require stronger semantics around what "completed successfully" means for a step file and when a rewind becomes mandatory.

The main open design questions are:

- whether moving from one `steps.N.sh` to the next should happen immediately or only after explicit user confirmation
- how the player should represent "stopped between files" versus "stopped in the middle of a file"
- how much of the recovery logic should be inferred from the event log versus materialized into a separate session state file

This is intentionally documented here before implementation so that tape authors understand the direction of the session model and the constraints that future step design may need to respect.

## 10.2. Remove the `.sh` Suffix from Docker Wrappers

Another planned usability improvement is to remove the `.sh` suffix from `docker/run.sh` and `docker/build.sh`. The goal is to make those wrappers easier to type, easier to remember, and closer to the syntax of the standard `docker` command itself.

That change would likely produce wrapper commands such as:

- `./docker/run play <steps-dir> [N]`
- `./docker/run rewind <steps-dir> [N]`
- `./docker/build [--all] [<steps-dir>]`

If this change is implemented, the whole documentation set must be updated so that examples, diagrams, tests, and private notes all reflect the new command names consistently.

# 11. Author (Paulo Jerônimo)

# 12. License

MIT License

Copyright (c) 2026 Paulo Jerônimo (https://paulojeronimo.com)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

# 13. Development model

This project is developed using a Cathedral of open source development.

In practice, this means I maintain two Git repositories:

- a private repository, where development happens with granular commits, experiments, and auxiliary scripts used for development workflows (for example, sync scripts)
- a public repository (the one users normally see on GitHub), used mostly to publish releases

This project is developed by only me (Paulo Jerônimo) and my AI agents.

Even though the development is led by a single person, contributions are welcome through GitHub pull requests.

## 13.1. References

- The Cathedral and the Bazaar:
  https://en.wikipedia.org/wiki/The_Cathedral_and_the_Bazaar

- Development Containers / GitHub Codespaces video:
  TBD